

CS 170 Proficiency Quiz 5

For this quiz you will write two short methods that **take arguments** and **return values**. The methods may require you to:

- ✓ **Parse** strings and convert them to integers or doubles
- ✓ **Format** numbers as strings, (using **String.format()**).
- ✓ Know how to **extract characters and substrings** from a **String**
- ✓ Know how to **find the position** of characters and substrings within a **String**
- ✓ Know how to use **concatenation** to produce new strings.
- ✓ Know how to **replace** a portion of a **String**

You will not use **printf()**; **there is no printing**. You will not need **if** statements, loops or **Scanners** for these problems.

You have 45 minutes to complete the problem. You may not access any notes (other than the Java Quick Reference below), files, or communicate with anyone in any way (including checking your email or looking up files on the Q: drive.) When you have finished and submitted the quiz, check to see that I have received it.

You can check your problem by using **Tools->Compile** to see if your code is syntactically correct. To **test your code**, use **Tools->Go**. You can **run** the program (in the console) by using **Tools->Run in Console**.

String Parsing, Formatting and Functions

The first method will be documented written inside **P01.java** and the second will be placed inside **P02.java**. Make sure you complete both methods. When you use **Tools->Go** it will test both methods. When you do **Tools->Run in Console** it will run the **main()** method in each class. You may add any additional code you like to the **main()** method. It is not part of the test, but it must compile.

Make sure you can successfully run Tools->Go before you submit your exam.
The testing done on your machine is submitted for your grade.

Variable and Constant Declarations

Type **Name** **Initial value**
/
/
/
int cansPerPack = 6;

final double CAN_VOLUME = 0.335;

Method Declaration

Modifiers **Return type** **Parameter type and name**
/
/
/
/
public static double cubeVolume(double sideLength)
{
 double volume = sideLength * sideLength * sideLength;
 return volume;
}

Exits method and returns result.

Conditional Statement

Condition
/
if (floor >= 13)
{
 actualFloor = floor - 1;
}
else if (floor >= 0) Second condition (optional)
{
 actualFloor = floor;
}
else
{
 System.out.println("Floor negative");
}
}

Executed when condition is true

Executed when all conditions are false (optional)

Loop Statements

Condition
/
while (balance < TARGET)
{
 year++;
 balance = balance * (1 + rate / 100);
}

Executed while condition is true

Initialization **Condition** **Update**
/
/
/
for (int i = 0; i < 10; i++)
{
 System.out.println(i);
}

Loop body executed at least once
do
{
 System.out.print("Enter a positive integer: ");
 input = in.nextInt();
}
while (input <= 0);

Set to a new element in each iteration

An array or collection
for (double value : values)
{
 sum = sum + value;
}

Executed for each element

String Operations

```
String s = "Hello";  
int n = s.length(); // 5  
char ch = s.charAt(1); // 'e'  
String t = s.substring(1, 4); // "ell"  
String u = s.toUpperCase(); // "HELLO"  
if (u.equals("HELLO")) ... // Use equals, not ==  
for (int i = 0; i < s.length(); i++)  
{  
    char ch = s.charAt(i);  
    Process ch  
}
```

Mathematical Operations

Math.pow(x, y)	Raising to a power x^y
Math.sqrt(x)	Square root $\sqrt{x}$
Math.log10(x)	Decimal log $\log_{10}(x)$
Math.abs(x)	Absolute value $ x $
Math.sin(x)	} Sine, cosine, tangent of x (x in radians)
Math.cos(x)	
Math.tan(x)	

Class Declaration

```
public class CashRegister  
{  
    private int itemCount;  
    private double totalPrice;  
}  
  
public void addItem(double price)  
{  
    itemCount++;  
    totalPrice = totalPrice + price;  
}  
...  
}
```

Instance variables

Method

Selected Operators and Their Precedence

(See Appendix B for the complete list.)

[]	Array element access
++ -- !	Increment, decrement, Boolean <i>not</i>
* / %	Multiplication, division, remainder
+ -	Addition, subtraction
< <= > >=	Comparisons
== !=	Equal, not equal
&&	Boolean <i>and</i>
	Boolean <i>or</i>
=	Assignment

Input

```
Scanner in = new Scanner(System.in);
// Can also use new Scanner(new File("input.txt"));

int n = in.nextInt();
double x = in.nextDouble();
String word = in.next();
String line = in.nextLine();

while (in.hasNextDouble())
{
    double x = in.nextDouble();
    Process x
}
```

Output

```
System.out.print("Enter a value: ");

System.out.println("Volume: " + volume);

System.out.printf("%-10s %10d %10.2f", name, qty, price);

PrintWriter out = new PrintWriter("output.txt");

out.close();
```

Does not advance to new line.

Use + to concatenate values.

Field width **Precision**

Left-justified string **Integer** **Floating-point number**

Use print/println/printf to write output to file.

Remember to close output file.

Arrays

```
int[] numbers = new int[5];
int[] squares = { 0, 1, 4, 9, 16 };
int[][] magicSquare =
{
    { 16, 3, 2, 13},
    { 5, 10, 11, 8},
    { 9, 6, 7, 12},
    { 4, 15, 14, 1}
};

for (int i = 0; i < numbers.length; i++)
{
    numbers[i] = i * i;
}

for (int element : numbers)
{
    Process element
}

System.out.println(Arrays.toString(numbers));
// Prints [0, 1, 4, 9, 16]
```

Element type **Element type** **Length**

All elements are zero.

Array Lists

```
ArrayList<String> names = new ArrayList<String>();

names.add("Ann");
names.add("Cindy"); // [Ann, Cindy], names.size() is now 2

names.add(1, "Bob"); // [Ann, Bob, Cindy]
names.remove(2); // [Ann, Bob]
names.set(1, "Bill"); // [Ann, Bill]

String name = names.get(0); // Gets "Ann"
System.out.println(names); // Prints [Ann, Bill]
```

Initially empty

Use wrapper type, Integer, Double, etc., for primitive types.

Element type

Add elements to the end

Linked Lists, Sets, and Iterators

```
LinkedList<String> names = new LinkedList<String>();
names.add("Bob"); // Adds at end

ListIterator<String> iter = names.listIterator();
iter.add("Ann"); // Adds before current position

String name = iter.next(); // Returns "Ann"
iter.remove(); // Removes "Ann"

Set<String> names = new HashSet<String>();
names.add("Ann"); // Adds to set if not present
names.remove("Bob"); // Removes if present

Iterator<String> iter = names.iterator();
while (iter.hasNext())
{
    Process iter.next()
}
```

Maps

```
Map<String, Integer> scores = new HashMap<String, Integer>();

scores.put("Bob", 10);
Integer score = scores.get("Bob");

for (String key : scores.keySet())
{
    Process key and scores.get(key)
}
```

Key type **Value type**

Returns null if key not present